

Working with Time Series

Python Data Science Handbook — Chapter 3, Section 3.11

Timestamps, periods, and durations — and the indexing, resampling, and rolling machinery built on top of them.

1 Why time deserves its own machinery

Background & Context

Time is the one “label” almost every real dataset carries: a sensor reading, a stock tick, a server log line, a patient visit. You could in principle store dates as plain strings and reason about them by hand — but the moment you want to ask “give me every row in July 2021,” “average this by month,” or “smooth this over a 30-day window,” string handling collapses. You need an index that *understands* that 2021-07-31 is followed by 2021-08-01, that a month is not a fixed number of days, and that subtracting two dates yields a *duration*.

Python’s story here is a layered one, and it helps to know the layers because Pandas stitches them together:

- **The standard library: `datetime` (and `dateutil`).** Python ships a `datetime` type, and the third-party `dateutil` adds flexible parsing (“4th of July, 2021” just works). This stack is wonderfully flexible but *slow*: each date is a full Python object, so a column of a million dates is a million-pointer object array.
- **NumPy’s `datetime64`.** NumPy encodes a date as a 64-bit integer (a count of time units from a fixed epoch). This is fast, typed, and vectorizes like any other NumPy array — but it is *rigid*: one fixed unit per array, and none of the friendly parsing or calendar logic.

Pandas unifies the two. It gives you a `Timestamp` object that has the *convenience* of `datetime` but is built on the *speed* of `datetime64`, and — crucially — a `DatetimeIndex` that turns an axis of timestamps into a first-class, sliceable, groupable index.

```
import numpy as np
import pandas as pd
```

Python Refresher

A 64-bit integer counting fixed-size units from an *epoch* (the reference instant 1970-01-01 T00:00:00) is exactly how Unix “epoch time” works. `datetime64` generalizes it: the same integer can count *nanoseconds*, *days*, *years*, etc. The unit is part of the dtype, written like `datetime64[ns]` (nanoseconds) or `datetime64[D]` (days). Pandas standardizes on nanosecond resolution.

2 The three kinds of time data

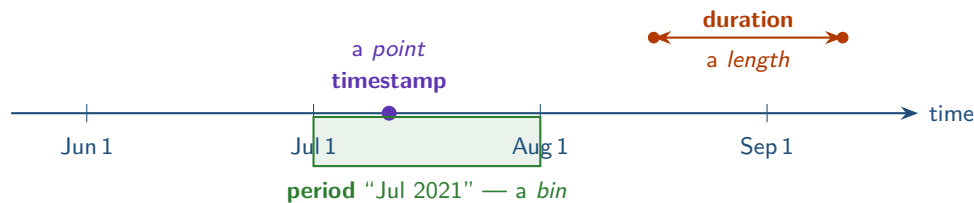
Before any API, fix the vocabulary. There are three conceptually distinct things you might mean by “a time,” and Pandas gives each its own scalar type *and* its own index type.

Concept	Meaning	Scalar class	Index class
Time <i>stamp</i>	a single instant (a point)	Timestamp	DatetimeIndex
Time <i>period</i>	a fixed-frequency interval (a bin)	Period	PeriodIndex
Time <i>duration</i>	an elapsed span (a length)	Timedelta	TimedeltaIndex

Intuition

The cleanest way to keep these straight is geometric — think of a timeline:

- A **timestamp** is a *point* on the line: “3:00 pm on July 4th.”
- A **period** is a *labeled interval* of fixed frequency: “the month of July 2021” — it has an anchor and a width, and consecutive periods tile the line without gaps.
- A **duration** is a *length* with no fixed position: “90 minutes.” Subtract two timestamps and you get one.



2.1 Creating each, by hand

```
ts = pd.Timestamp('2021-07-04 15:00') # a point in time
pr = pd.Period('2021-07', freq='M')    # the whole month of July 2021
td = pd.Timedelta(days=90)             # a 90-day length

ts2 = pd.Timestamp('2021-10-02 15:00')
ts2 - ts                                # subtracting points → a Timedelta
```

```
Timedelta('90 days 00:00:00')
```

Note the arithmetic that ties them together: `Timestamp - Timestamp = Timedelta`, and `Timestamp + Timedelta = Timestamp`. Periods behave like calendar bins: `pr + 1` is August 2021, and `ts in pr` tests whether the instant falls inside the bin.

2.2 Converting between the three

These types are not islands; you move between them constantly, and knowing the conversions saves a lot of fumbling:

- **Stamp** → **period** with `.to_period(freq)`: “which bin does this instant fall in?” A `DatetimeIndex` of daily readings becomes a `PeriodIndex` of months via `idx.to_period('M')` — handy right before a group-by-month.
- **Period** → **stamp** with `.to_timestamp(how=...)`: collapse a bin back to a point, anchoring at its `'start'` or `'end'`.

- **Stamp** → **duration** by subtraction, as above; and a `DatetimeIndex` minus a single `Timestamp` yields a `TimedeltaIndex` of elapsed times — the natural “time since launch” axis.

```
p = pd.Timestamp('2021-07-04 15:00').to_period('M') # Period('2021-07', 'M')
p.to_timestamp(how='start') # → Timestamp('2021-07-01 00:00:00')
p.to_timestamp(how='end') # → Timestamp('2021-07-31 23:59:59.999...')
```

Notice the asymmetry: a stamp carries *more* information than the period it maps to (the exact instant), so `to_period` *discards* detail, while `to_timestamp` has to *invent* a convention (start vs. end) to recover a point. Conversions that lose information are one-way — a recurring theme worth flagging to yourself whenever you reach for one.

3 Constructors: building indexes in bulk

You rarely create one timestamp at a time. The workhorses are `pd.to_datetime` (parse whatever you have into timestamps) and the `*_range` family (manufacture a regular grid).

3.1 Parsing: `pd.to_datetime`

`pd.to_datetime` is the flexible front door. Hand it a single string and you get a `Timestamp`; hand it a list/array/column and you get a `DatetimeIndex`. It understands a remarkable range of formats:

```
pd.to_datetime('4th of July, 2021') # → Timestamp('2021-07-04 00:00:00')
dates = pd.to_datetime(['2021-07-04', '2021-07-05', '2021-07-08'])
print(dates)
```

```
DatetimeIndex(['2021-07-04', '2021-07-05', '2021-07-08'],
              dtype='datetime64[ns]', freq=None)
```

Common Pitfall

Day-first vs. month-first ambiguity. A string like `'04/07/2021'` is genuinely ambiguous: is it April 7th or July 4th? `to_datetime` defaults to *month-first* (US style) and will silently pick July... sorry, *April*. If your data is day-first, pass `dayfirst=True`. Better still, when you know the exact layout, pass an explicit `format='%d/%m/%Y'`: it is both unambiguous *and* much faster, because it skips the format-sniffing step on every value.

3.2 Regular grids: the `*_range` family

To manufacture a regular sequence, choose the constructor that matches the *kind* of data you want:

```
pd.date_range('2021-07-01', periods=5, freq='D') # 5 daily timestamps
pd.date_range('2021-07-01', '2021-07-10', freq='2D') # every 2nd day, by endpoints
pd.period_range('2021-01', periods=4, freq='M') # 4 monthly periods
pd.timedelta_range(0, periods=4, freq='6H') # 0h, 6h, 12h, 18h
```

```
PeriodIndex(['2021-01', '2021-02', '2021-03', '2021-04'],
            dtype='period[M]')
```

You specify a range two ways: `start + periods` (a count), or `start + end` (by endpoints). The `freq` argument controls the spacing, and it is worth memorizing the common codes.

3.3 Frequency and offset codes

Code	Meaning	Code	Meaning
D	calendar day	H	hour
B	business day (Mon–Fri)	T/ min	minute
W	week (Sun-anchored)	S	second
M	month <i>end</i>	MS	month <i>start</i>
Q	quarter end	QS	quarter start
A/Y	year end	AS/YS	year start

Key Idea

Two patterns make the code list far more powerful than it looks:

- **Numbers multiply.** Prefix a code with an integer to scale it: `'2D'` is every other day, `'3M'` is every third month-end.
- **Codes concatenate (add).** Stack codes to combine units: `'2H30T'` is a 2-hour-30-minute step, `'1D6H'` is 30 hours.

And mind the S suffix: a bare M, Q, A anchors to the *end* of the period (so `date_range(freq='M')` lands on Jan 31, Feb 28, ...), while MS, QS, YS anchor to the *start* (Jan 1, Feb 1, ...). Picking the wrong one is a classic off-by-a-month bug in monthly reports.

4 Indexing by time

The payoff for all this typing is that a `DatetimeIndex` makes time-based selection feel effortless. Put timestamps on the index of a `Series`, and you unlock *partial-string indexing*.

```
idx = pd.to_datetime(['2020-07-04', '2020-08-04',
                      '2021-07-04', '2021-08-04'])
data = pd.Series([0, 1, 2, 3], index=idx)
print(data)
```

```
2020-07-04    0
2020-08-04    1
2021-07-04    2
2021-08-04    3
dtype: int64
```

Now you can select by a *partial* date string, and Pandas interprets it as “every timestamp that falls

inside this calendar span”:

```
data['2021']          # everything in the year 2021 → the last two rows
data['2020-07']       # everything in July 2020      → the first row
data['2020-08':'2021-07'] # an inclusive date-range slice
```

```
2021-07-04    2
2021-08-04    3
dtype: int64
```

Intuition

Why does `data['2021']` return *rows* when `df['col']` on a regular `DataFrame` returns a *column*? Because a `DatetimeIndex` teaches the index how to “parse” a string into a span and match it against the row labels. `'2021'` is not a column name — it is a coarse timestamp that resolves to the half-open interval `[2021-01-01, 2022-01-01)`, and every label inside it comes back. This is the single most ergonomic feature of time-series Pandas.

5 A worked synthetic series

To make resampling and rolling concrete, we need a realistic daily series. We build one with a *fixed seed* so the figures below are reproducible: a cumulative-sum random walk (the slow drift) plus a yearly sine term (the season) plus a little daily noise, indexed by a two-year `date_range`.

```
rng = np.random.default_rng(0)
dates = pd.date_range('2021-01-01', periods=730, freq='D')
seasonal = 8.0 * np.sin(2 * np.pi * dates.dayofyear / 365.0)
walk = np.cumsum(rng.normal(0.02, 1.0, size=len(dates))) # the drift
noise = rng.normal(scale=1.5, size=len(dates))
series = pd.Series(50 + walk + seasonal + noise,
                  index=dates, name='signal')
```

6 Resampling and frequency conversion

A daily series is often *too* granular. Two distinct operations change its frequency, and conflating them is a common mistake.

6.1 resample: a groupby over time bins

`resample` is a *reduction*. It chops the index into bins of the target frequency and applies an aggregation to each bin — it is, almost literally, “`groupby` where the grouping key is the time bin.”

```
monthly = series.resample('M').mean() # mean of each calendar month
monthly.head(3)
```

```
2021-01-31    50.83
2021-02-28    52.11
2021-03-31    54.02
```

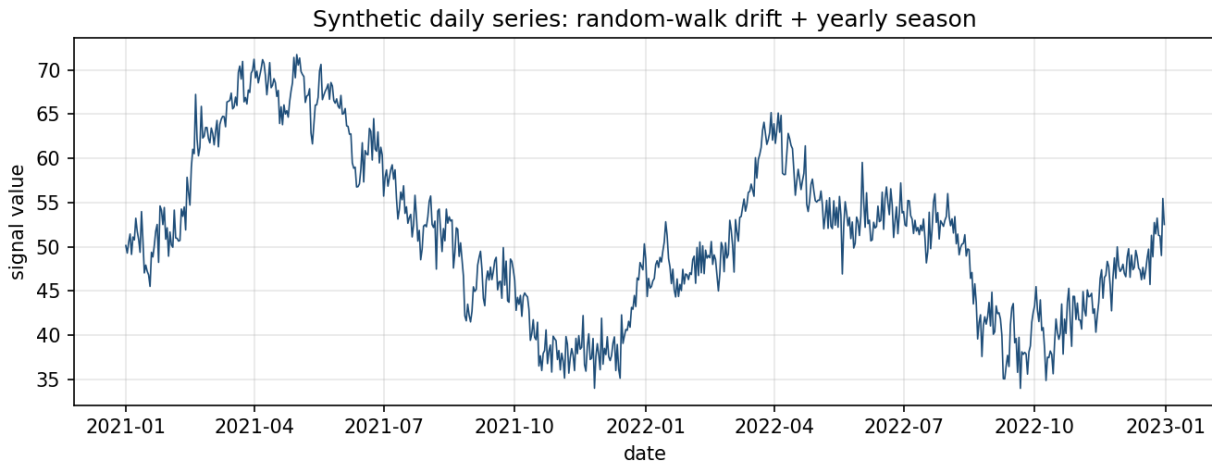


Figure 1: The raw synthetic daily series. The slow up-and-down wander is the random-walk *drift*; the gentle yearly bulge is the *seasonal* sine term; the fuzz around the line is daily noise. Two years of daily data is 730 points — dense enough that the underlying structure is hard to read by eye, which is exactly what motivates the smoothing tools below.

```
Freq: M, Name: signal, dtype: float64
```

Because it aggregates, `resample` naturally handles **down-sampling** (going to a *coarser* frequency, e.g. daily $\rightarrow$ monthly): each output point summarizes many input points. You can use any reduction — `.sum()`, `.max()`, `.std()`, `.ohlc()` for finance-style open/high/low/close, or `.agg([...])` for several at once:

```
series.resample('Q').agg(['mean', 'min', 'max']) # one row per quarter
```

	mean	min	max
2021-03-31	51.6	46.9	57.1
2021-06-30	58.2	52.0	63.8
...			

The resulting frame has one row per quarter and three columns — exactly the shape of a summary report. This is why `resample` is the backbone of nearly every “roll my hourly logs up to a daily dashboard” pipeline.

6.2 asfreq: pure point selection

`asfreq` does *not* aggregate. It simply changes the frequency of the index and keeps (or looks up) the value exactly at each new timestamp. On **up-sampling** (going *finer*, e.g. monthly $\rightarrow$ daily) there are new timestamps with no data, so you must say how to fill the holes:

```
monthly.asfreq('D') # new daily slots → NaN where no data
monthly.asfreq('D', method='ffill') # forward-fill: carry last value
forward
```

```
monthly.asfreq('D', method='bfill') # back-fill: pull next value backward
```

Key Idea

The one-line distinction. `resample` *summarizes* (it reduces many points to one per bin — think “aggregate”); `asfreq` *selects/relabels* (it keeps the value sitting at each new timestamp — think “reindex onto a new grid”). For down-sampling you almost always want `resample`; for up-sampling you want `asfreq` (or `resample(...).interpolate()`), and you must choose a fill rule (`ffill/bfill`) because new, empty slots are created.

Common Pitfall

`resample('M')` labels each bin by its *right* (end) edge — January’s mean is stamped 2021-01-31. If you intend bins labeled at the start of the month, use `resample('MS')` or pass `label='left'`. Silently mislabeled bins shift every downstream join by one period.

7 Rolling-window statistics

Resampling collapses the time axis; a *rolling window* keeps it. `.rolling(window)` creates a sliding view, and you reduce within each window to get a smoothed series *at the original frequency*.

```
roll = series.rolling(window=30, center=True)
smooth = roll.mean()           # 30-day moving average
band   = roll.std()           # 30-day moving volatility
```

Intuition

Picture a 30-day window gliding one day at a time along the series. At each position it averages the 30 values it currently covers and emits one number. The result has the *same* number of points as the input (so you can overlay it directly), but the day-to-day jitter is averaged away, leaving the slow drift and the season visible. `center=True` aligns each output to the *middle* of its window rather than the trailing edge, so the smoothed curve does not lag the data — the trade-off is that you cannot compute it in real time, since it peeks at future points.

The figure below contrasts both smoothers on our synthetic series: the faint raw line, a coarse monthly `resample().mean()`, and a 30-day `rolling().mean()`.

7.1 When to reach for which

The three tools answer different questions, and a quick decision rule keeps them straight. Ask *what the output should look like*:

You want...	Tool	Output frequency
fewer points, one summary per bin	<code>resample(f).agg</code>	coarser than input
the same value on a new grid	<code>asfreq(f)</code>	finer or coarser (no aggregation)
a smooth trend at full resolution	<code>rolling(w).agg</code>	same as input

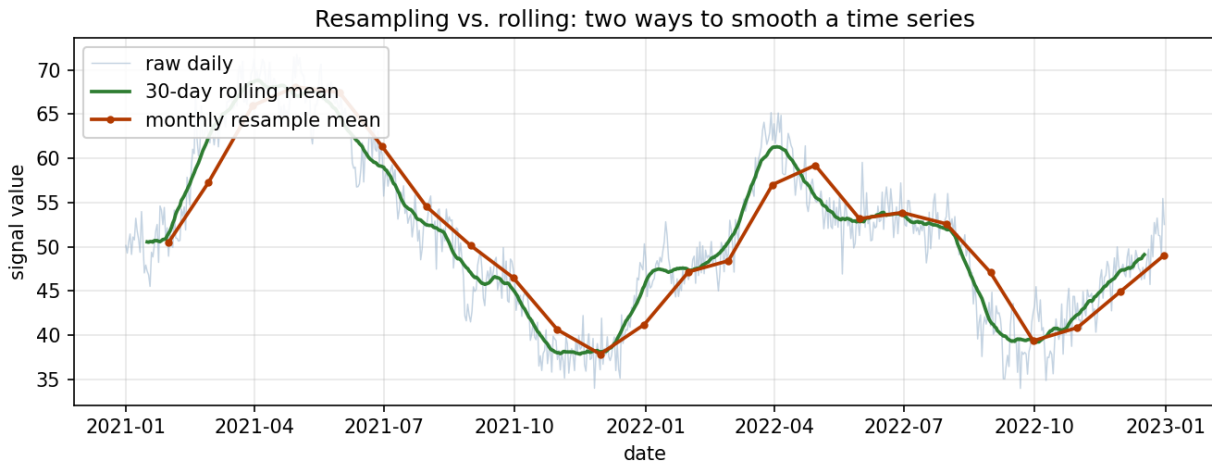


Figure 2: Two ways to smooth. The pale line is the raw daily series. The orange markers are the monthly `resample('M').mean()` — one point per month, sitting at month-end, a sparse summary. The green curve is the 30-day `rolling().mean()` — a dense, continuous smoother at the original daily frequency. Both reveal the drift-plus-season structure that the raw fuzz hides; `resample` gives you fewer, calendar-aligned points, while `rolling` gives you a full-resolution trend line.

A useful sanity check: `resample` and `asfreq` *change the number of rows* (the index frequency changes); `rolling` *preserves* it (only the values change, smoothed in place). If a smoothing step unexpectedly shrank your frame, you reached for the wrong one.

Python Refresher

A rolling mean is a discrete *convolution* of the series with a uniform window of weights $1/w$. That framing tells you why the edges are special: near the start and end there are fewer than w neighbours, so by default those positions come back as NaN (pass `min_periods` to relax this). It also hints at the richer cousins you will meet later: `.ewm()` (exponentially weighted, so recent points count more) and windowed `.corr()/.cov()` for moving relationships between two series.

Section recap

- Python's time stack: `datetime/dateutil` (flexible, slow, object arrays) and NumPy `datetime64` (fast, typed, rigid). Pandas fuses them as `Timestamp + DatetimeIndex`.
- Three kinds of time data: **stamps** (`Timestamp/DatetimeIndex`, a point), **periods** (`Period/PeriodIndex`, a fixed-frequency bin), **durations** (`Timedelta/TimedeltaIndex`, a length). Stamp – stamp = duration.
- Build with `pd.to_datetime` (flexible parsing; watch day/month ambiguity) and `date_range/period_range/timedelta_range`. Frequency codes scale with a number (`'2D'`) and concatenate (`'2H30T'`); MS/QS/YS anchor at the start, bare M/Q/A at the end.
- A `DatetimeIndex` enables partial-string indexing: `data['2021']`, `data['2021-07']`, and inclusive date-range slices.

- `resample` = groupby-over-time-bins reduction (down-sampling); `asfreq` = point selection onto a new grid (up-sampling needs `ffill/bfill`).
- `.rolling(window)` smooths at the original frequency by sliding a window; use `center=True` to avoid lag.